

大连海洋大学

红色文化学习打卡系统设计与实现

学 生 姓 名:	李发鹏
指 导 教 师:	史鹏辉
合 作 指 导 教 师:	吴俊峰
专 业 名 称:	计算机科学与技术
所 在 学 院:	信息工程学院

2026 年 9 月

第一章 系统分析

1.1 业务流程分析

在梳理业务流程时，先从普通用户一次完整的学习操作入手。用户打开客户端后，需要先填写服务端地址并完成登录；没有账号时可以转到注册窗口，注册成功后再返回登录。进入主界面以后，用户可以浏览全部景点或热门景点，也可以通过编号和名称缩小查询范围。选定景点并填写学习心得后，系统检查心得是否为空，检查通过才保存学习记录并刷新积分。图 1.1 把这条主流程和注册失败、登录失败、景点不存在等异常分支放在同一张图中，能够较直观地看出用户从进入系统到完成一次打卡的实际操作顺序。

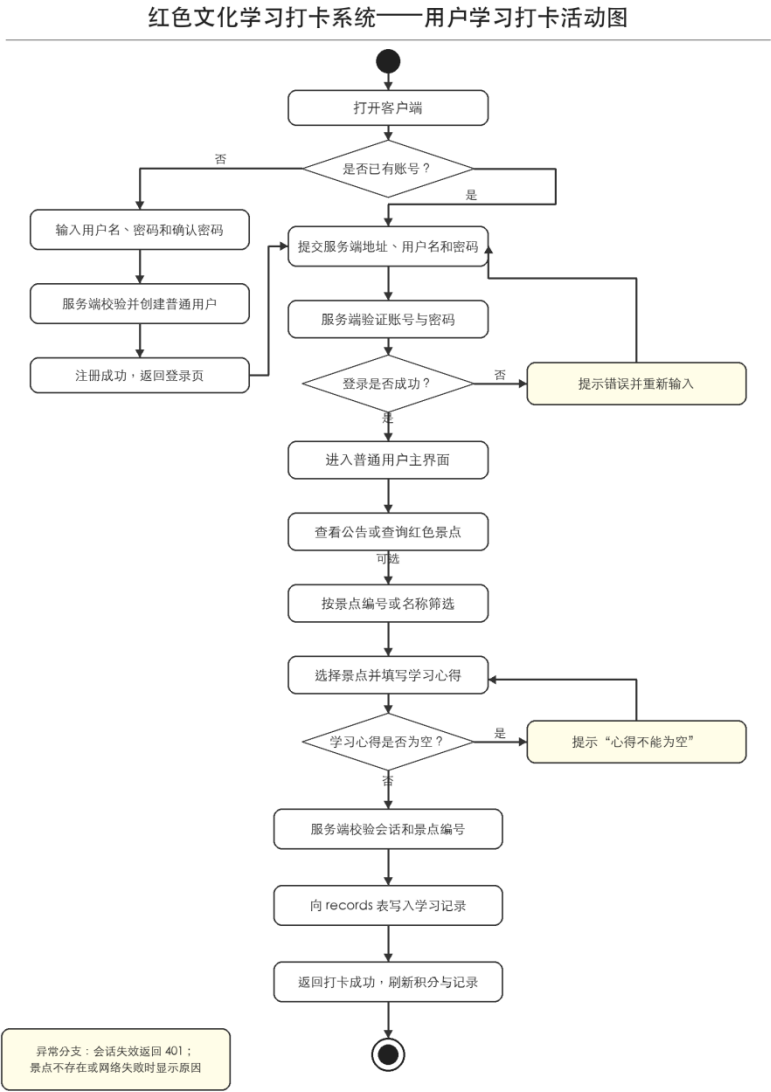


图 1.1 用户学习打卡活动图

1.2 事件列表与用例分析

根据上述业务流程，将参与者主动发起并产生明确结果的操作整理为业务事件。客户端负责接收输入和显示结果，服务端负责身份验证、权限判断和数据处理。主要业务事件见表 1.1。

表 1.1 业务事件列表

编号	参与者	业务事件	系统响应
E01	普通用户	注册账号	校验数据并创建普通用户
E02	用户/管理员	登录系统	验证身份并签发 8 小时会话令牌
E03	普通用户	查看或查询公告	返回符合编号或标题条件的公告
E04	普通用户	查看或查询景点	返回景点资料和当前打卡次数
E05	普通用户	查看热门景点	返回打卡次数不少于 10 次的景点
E06	普通用户	提交学习打卡	写入学习记录并返回积分增加提示
E07	普通用户	管理本人学习记录	查询、修改或删除本人记录
E08	普通用户	修改账号资料	验证原密码后更新账号并清除旧会话
E09	管理员	查看普通用户	返回用户及其实时积分
E10	管理员	维护公告	新增、修改、查询或删除公告
E11	管理员	维护景点	新增、修改、查询或删除景点
E12	用户/管理员	退出登录	删除当前会话令牌

业务事件列表解决了“系统要响应哪些操作”的问题，在此基础上还需要区分普通用户和管理员各自能使用哪些功能。普通用户主要完成账号注册、景点查询、学习打卡和个人记录管理；管理员使用同一入口登录，主要完成用户查看、景点维护和公告维护。图 1.2 采用较紧凑的布局保留主要用例，查询编号、输入校验等细节不再单独展开，避免用例图过长。图中的功能范围与表 1.1 中的业务事件保持对应。

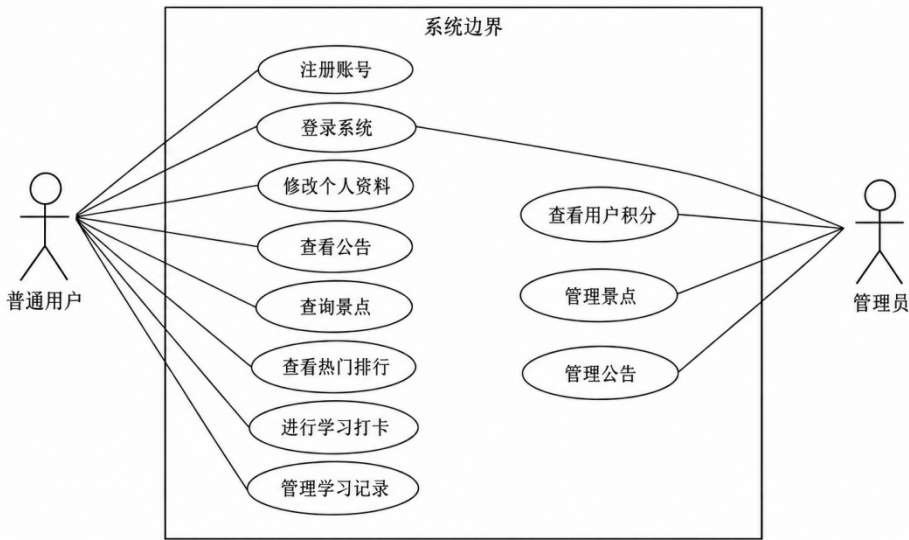


图 1.2 系统用例图

1.2.1 普通用户用例描述

普通用户的核心用例是学习打卡。该用例把景点选择、心得填写、记录保存和积分反馈连接为完整过程，具体描述见表 1.2。

表 1.2 学习打卡用例描述

项目	内容
用例名称	学习打卡
参与者	普通用户
前置条件	用户已登录且会话有效，目标景点存在
基本流程	选择景点，填写学习心得，提交打卡，服务端写入记录，客户端刷新积分和记录
异常流程	会话失效、景点不存在或心得为空时拒绝写入并显示原因
后置条件	records 表新增一条记录，用户积分和景点打卡次数的实时统计结果随之增加

1.2.2 管理员用例描述

管理员从统一登录入口进入系统，登录成功后可以维护景点和公告。服务端会再次检查管理员身份，避免普通用户通过直接发送请求执行管理操作。管理员内容维护用例见表 1.3。

表 1.3 管理员内容维护用例描述

项目	内容
用例名称	维护公告和景点
参与者	管理员
前置条件	使用服务端固定管理员账号登录且会话有效
基本流程	查看列表，选择新增或编辑，提交内容，服务端校验后写入数据库并返回结果
异常流程	字段为空、编号不存在或删除已有打卡记录关联的景点时拒绝操作
后置条件	公告或景点数据按请求完成新增、修改或删除

1.3 静态结构分析

根据前面的用例分析，我把系统中经常出现的对象整理为用户、学习记录、景点、管理员和公告，如图 1.3 所示。用户可以产生多条学习记录，一个景点也可以对应多条学习记录，因此学习记录处于用户与景点之间，保存每次打卡的时间和心得。管理员主要负责维护景点和公告。这里先列出各对象的基本属性及静态关系，不展开具体操作方法；学习打卡时各部分怎样配合，在后面的分析类图中再说明。

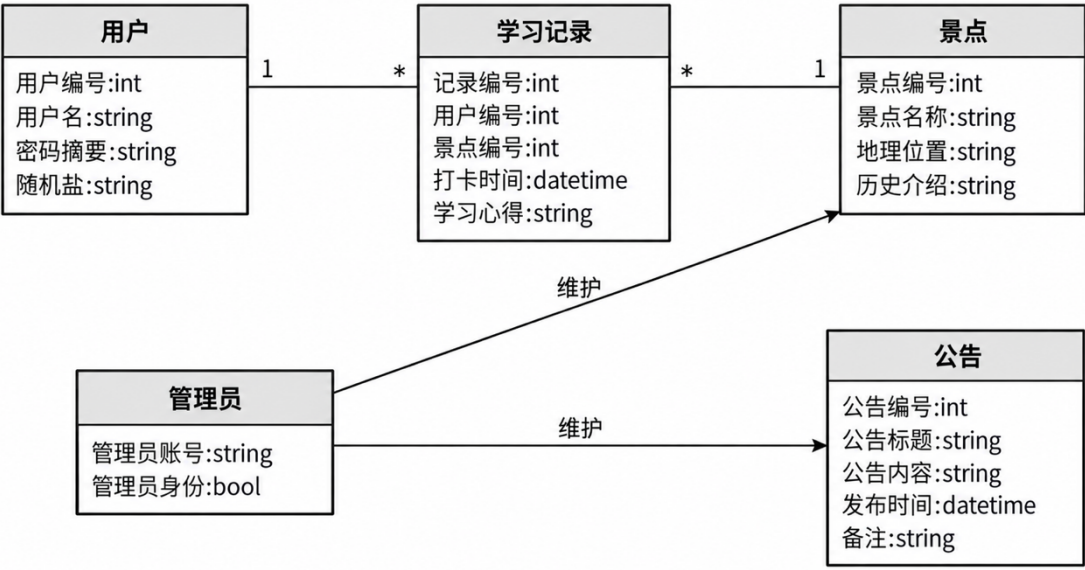


图 1.3 系统静态类图

1.4 动态交互分析

静态关系确定了系统中有哪些主要对象，接下来还需要说明一次打卡是怎样在各模块之间传递的。用户在 LearningApp 中选中景点并填写心得后，界面把景点编号和心得交给 ApiClient；ApiClient 将数据和登录令牌封装成 HTTP 请求，由服务端 Handler 解析，再交给 LearningService 检查会话、用户身份、景点是否存在以及心得内容。检查通过后，服务端向 records 表写入记录，并把处理结果逐层返回给客户端。客户端收到成功结果后刷新积分、景点打卡次数和学习记录。图 1.4 给出了这条交互链路，也为第二章中表示层、通信层、业务层和数据层的划分提供了依据。

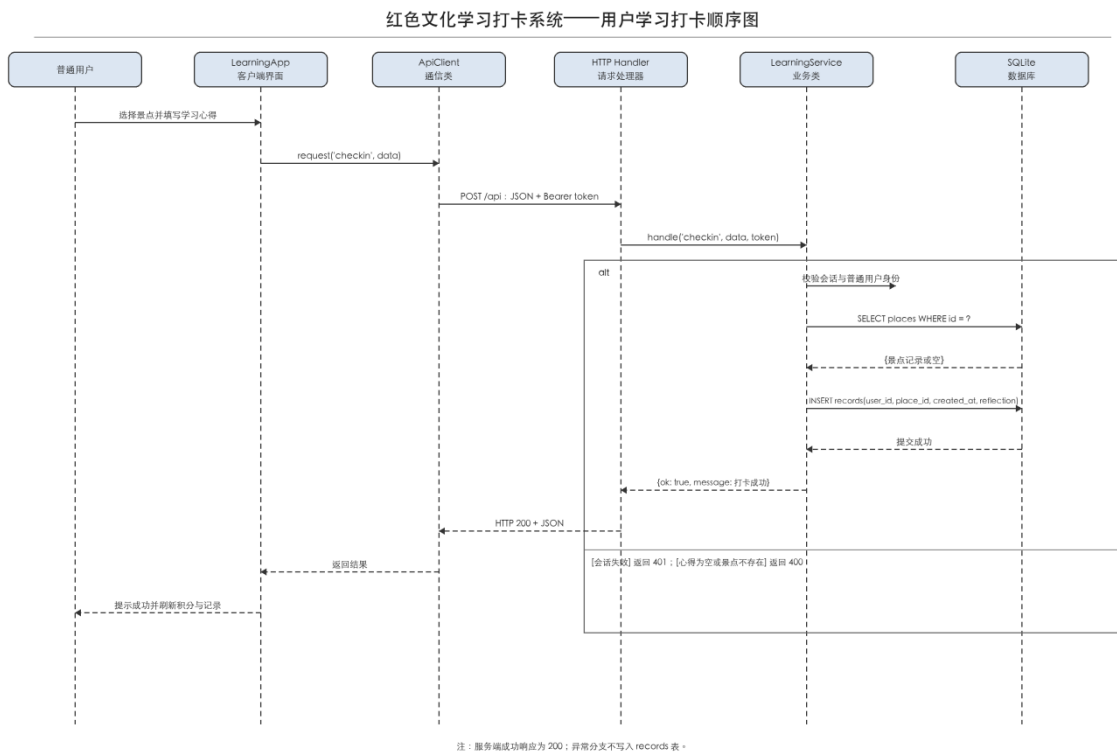


图 1.4 用户学习打卡顺序图

1.5 分析类识别

结合图 1.4 中的打卡顺序，我进一步把打卡过程涉及的职责分开，如图 1.5 所示。学习打卡页面负责让用户选择景点、填写心得并提交；打卡控制负责检查登录状态和景点是否存在，再创建学习记录；用户、景点和学习记录则保存或提供处理打卡所需的数据。图中的方法表示分析阶段对各类职责的概括，用来说明这个功能由哪些部分共同完成，具体程序中的函数和调用方式在第二章的类设计及第三章的实现中再展开。

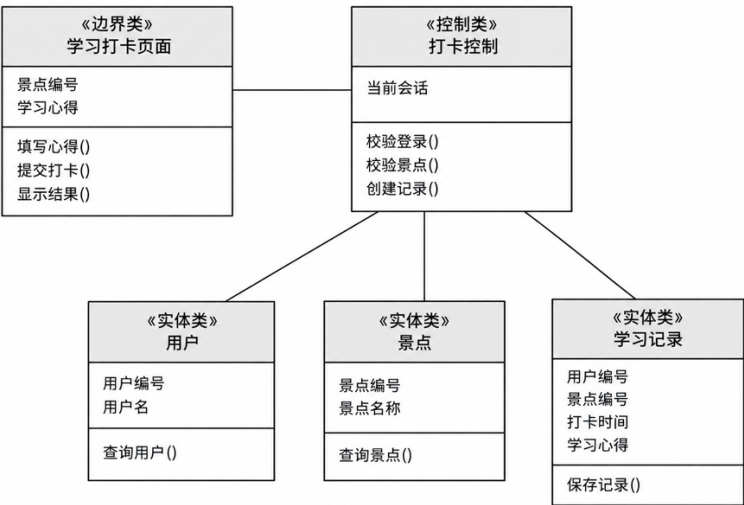


图 1.5 学习打卡分析类图

第二章 系统设计

2.1 体系结构模型

2.1.1 逻辑结构设计

为了使界面操作、网络通信和数据库处理不混在同一个文件中，系统按照实际代码职责划分为表示层、通信层、业务层和数据层。表示层由 Tkinter 和 ttk 构成，负责接收输入、展示列表和弹出提示；通信层中的 ApiClient 负责把 action 和 data 转成 JSON，并携带 Bearer 令牌访问接口；业务层由 HTTP 请求处理器和 LearningService 组成，分别完成请求解析、动作分发、会话检查和业务判断；数据层使用 SQLite 保存用户、景点、学习记录和公告。图 2.1 展示了请求由上向下处理、结果再由下向上返回的过程，后面的类设计也按照这一层次分配方法。

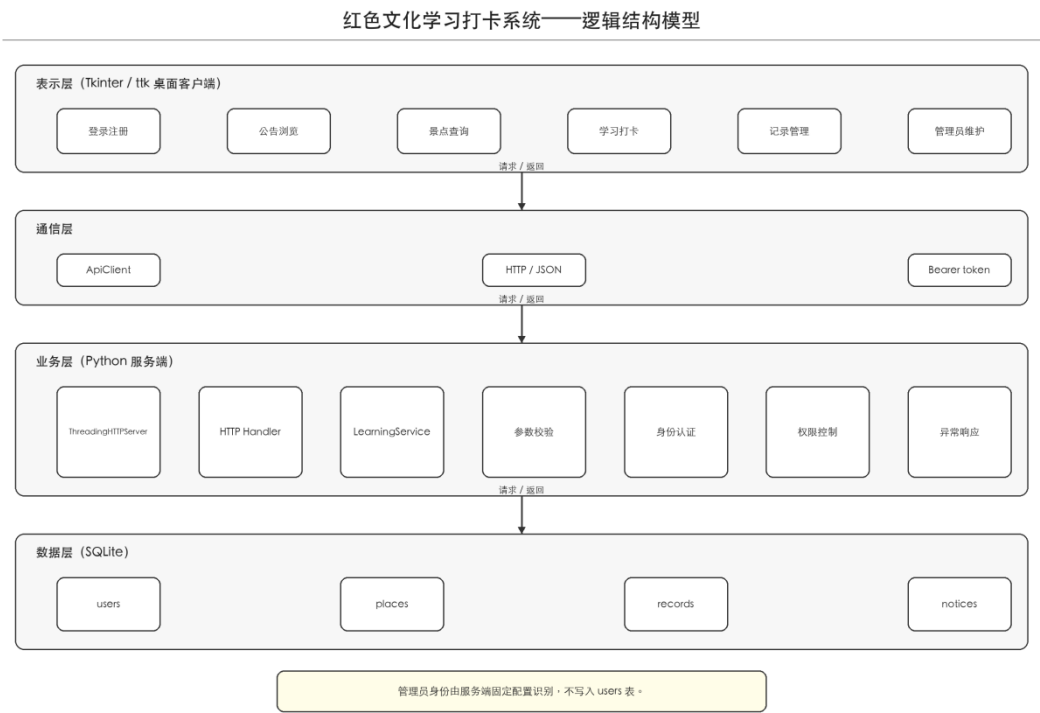
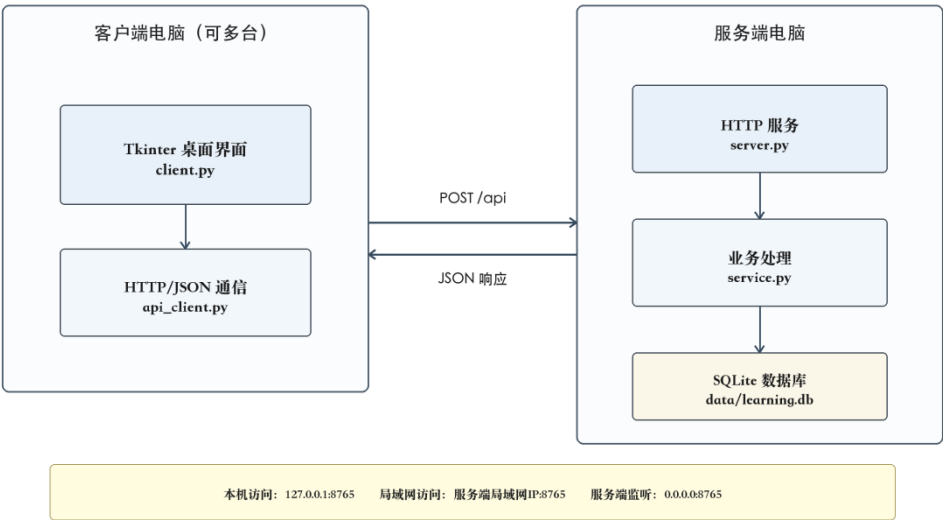


图 2.1 系统逻辑结构示意图

2.1.2 物理结构设计

逻辑结构落实到实际运行环境后，系统由客户端电脑和服务端电脑两部分组成。客户端运行 client.py，通过 api_client.py 发送 HTTP/JSON 请求，不直接接触数据库；服务端运行 server.py 和 service.py，并在服务端本地读写 data/learning.db。两部分既可以放在同一台电脑上完成课程演示，也可以分别放在同一局域网中的两台电脑上运行。本机使用 127.0.0.1:8765 访问，跨电脑时客户端改填服务端局域网 IP，服务端继续监听 0.0.0.0:8765。图 2.2 把程序文件、通信方向和数据库所在位置放在一起说明，可以看出多台客户端共享的是同一套服务端数据。

红色文化学习打卡系统物理结构



2.2 类的设计

系统类设计按照界面、通信、请求处理、业务逻辑和数据访问职责进行划分。LearningApp 负责界面和交互，ApiClient 负责网络请求，HTTP Handler 负责接收并分发接口请求，LearningService 负责业务规则，database 模块负责数据库初始化和连接。主要类及模块见表 2.1。

表 2.1 主要类与模块设计

类或模块	主要属性	主要方法	职责
LearningApp	root, api, tables, rows, busy	request, logged_in, load, checkin, edit_record, delete	构建桌面界面并组织交互
ApiClient	base_url, token	send, health, call	封装 HTTP/JSON 请求和异常
HTTP Handler	service	do_GET, do_POST, reply, reply_home	解析请求、分发动作和映射状态码
LearningService	db_path, sessions, session_lock	profile, handle	执行业务规则、会话和权限校验
database 模块	DEFAULT_DB	connect, initialize	建立 SQLite 连接、事务和表结构

第二章的类设计是在第一章领域对象分析的基础上进一步细化得到的。LearningApp 保存当前用户、页面表格和查询条件，并负责登录、列表加载和打卡窗口；ApiClient 保存服务地址和会话令牌，统一处理 HTTP 请求及连接异常；Handler 接收 GET 或 POST 请求并返回相应状态码；LearningService 保存数据库路径和会话信息，集中执行注册登录、内容查询、打卡和管理员操作；database 模块完成表结构初始化和数据库连接。图 2.3 列出了这些类和模块的主要属性、方法及调用关系，图中的方法名称与项目源代码保持一致，便于第三章结合页面功能说明具体实现。

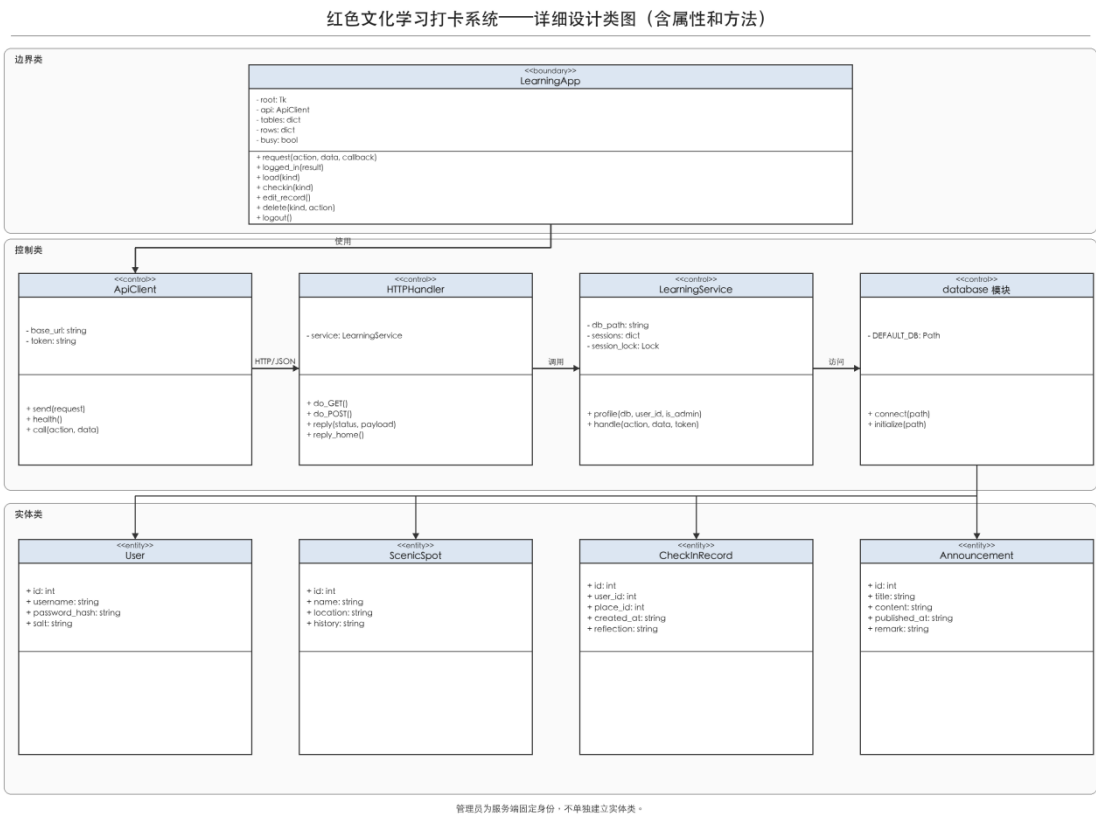


图 2.3 详细设计类图（含属性和方法）

2.3 数据库设计

2.3.1 实体属性设计

本节先对数据库中需要保存的用户、景点、学习记录和公告进行实体属性设计。实体属性图主要用于说明每个实体包含哪些业务属性，不在图中标注主键、外键和表之间的连接关系。图中的矩形表示实体，椭圆表示属性，实体与属性之间的连线直接连接到图形边界。主键、外键以及由多对多关系转换得到的学习记录表，将在后面的数据库逻辑设计中说明。

（1）用户实体。用户实体保存普通用户登录所需的信息，包括用户编号、用户名、密码摘要和随机盐。实体属性设计阶段只说明这些数据属于用户对象，不在图中添加主键或外键标记。管理员为服务端固定身份，不放入用户实体。用户实体属性见图 2.4。

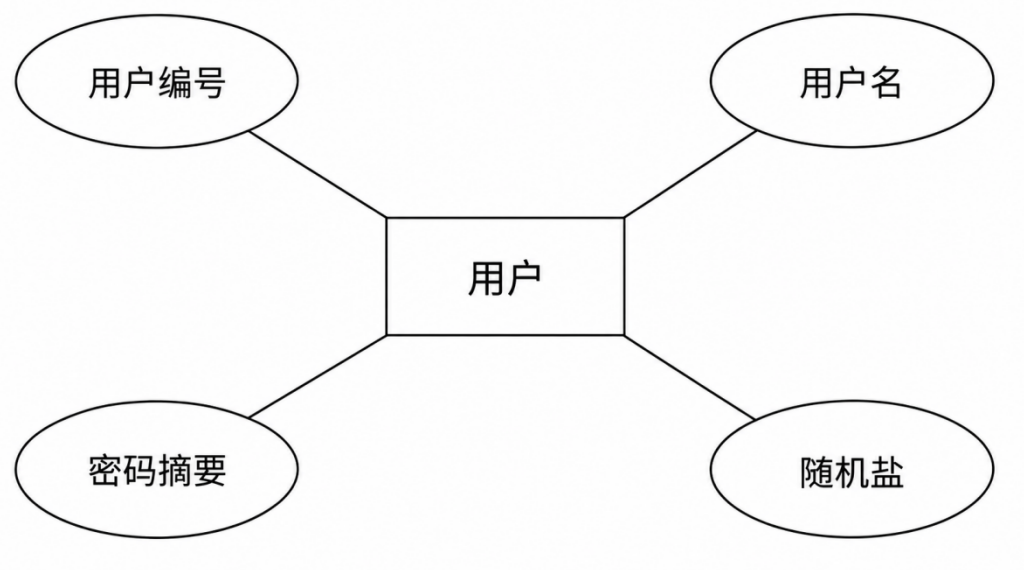


图 2.4 用户实体属性图

（2）景点实体。景点实体保存红色文化景点的编号、名称、地理位置和历史介绍，用于景点列表展示、条件查询和详情查看。景点打卡次数由学习记录统计得到，不作为景点的固定属性。景点实体属性见图 2.5。

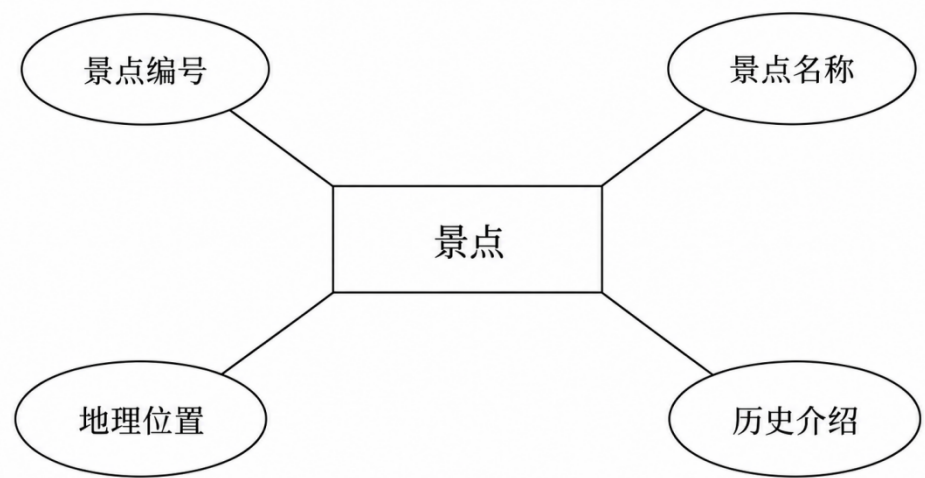


图 2.5 景点实体属性图

（3）学习记录实体。学习记录用于保存一次打卡形成的数据，包括记录编号、用户编号、景点编号、打卡时间和学习心得。本节只列出记录本身需要保存的属性，用户编号和景点编号的外键作用放到数据库逻辑设计中说明。学习记录实体属性见图 2.6。

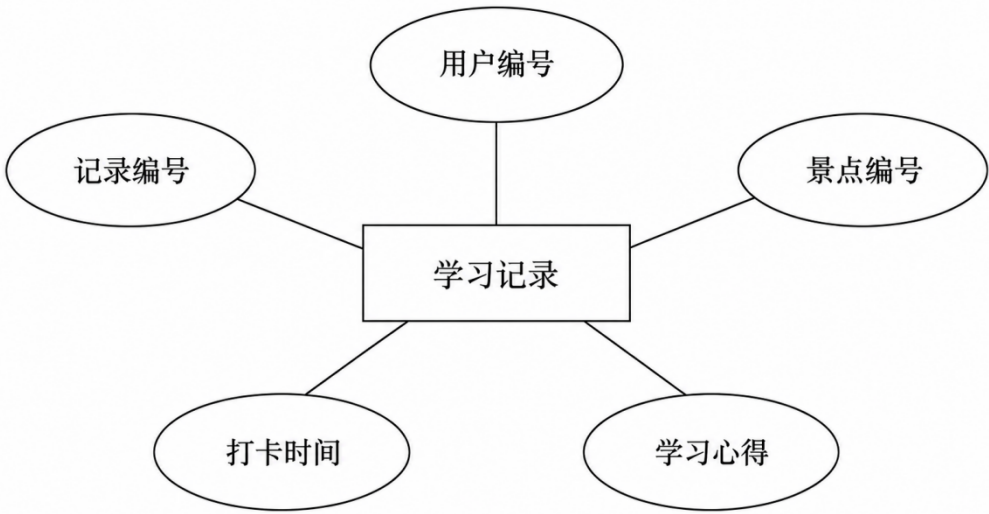


图 2.6 学习记录实体属性图

（4）公告实体。公告实体保存公告编号、公告标题、公告内容、发布时间和备注。公告由管理员功能维护，但它与用户、景点和学习打卡没有直接的数据关系，因此只单独说明实体属性，不放入后面的系统整体 E-R 关系图。公告实体属性见图 2.7。

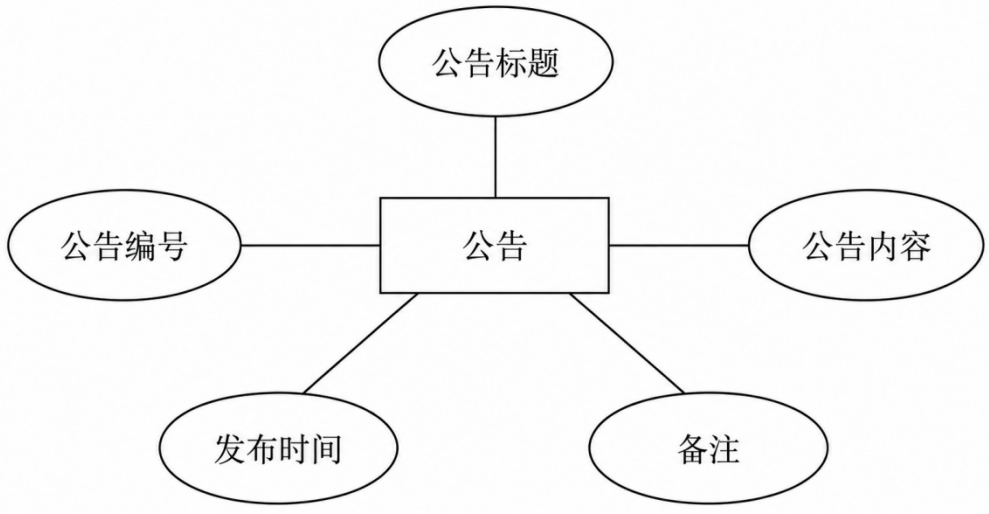


图 2.7 公告实体属性图

（5）系统整体 E-R 图。在业务概念层面，普通用户可以学习多个景点，同一个景点也可以被多名用户学习，因此用户和景点之间形成多对多的“学习打卡”联系。学习记录表是把这一多对多联系转换为关系数据库结构时产生的关联表，不作为整体概念 E-R 图中的独立实体；公告没有参与该联系，也不放入图中。图 2.8 只保留用户、学习打卡联系和景点，避免把逻辑设计阶段的主键、外键和关联表提前放入概要关系中。

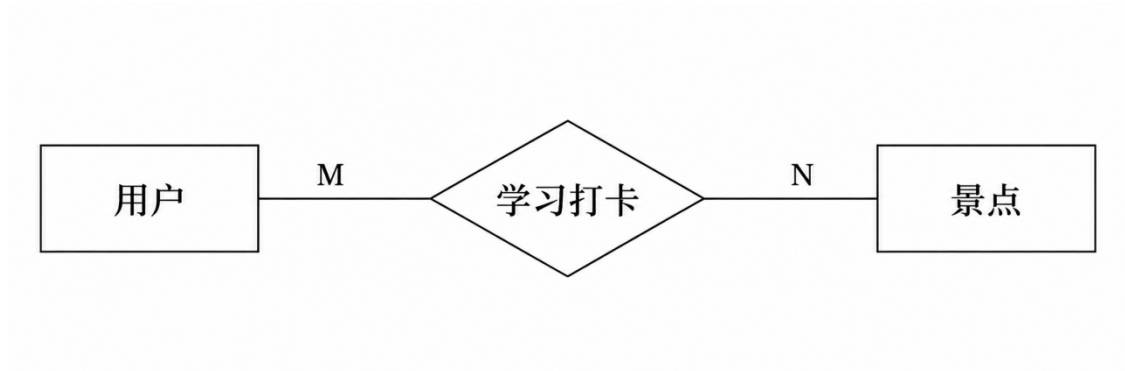


图 2.8 系统整体 E-R 图

2.3.2 数据库逻辑设计

根据实体属性设计和图 2.8 中的多对多联系，将数据库转换为 users、places、records 和 notices 四个关系模式。users 表保存普通用户账号，places 表保存景点资料；用户与景点之间的“学习打卡”联系转换为 records 关联表，并在其中加入用户编号和景点编号；notices 表独立保存公告内容。以下关系模式中，主键使用直线标出，外键使用波浪线标出。

- 用户（用户编号，用户名，密码摘要，随机盐）。主键为用户编号，用户名具有唯一约束。
- 景点（景点编号，景点名称，地理位置，历史介绍）。主键为景点编号。
- 学习记录（记录编号，用户编号，景点编号，打卡时间，学习心得）。主键为记录编号，用户编号和景点编号为外键。
- 公告（公告编号，公告标题，公告内容，发布时间，备注）。主键为公告编号。

2.3.3 数据表结构

（1）users 表。users 表用于保存普通用户的账号信息。id 为自增主键，username 不能为空且不能重复，password_hash 和 salt 共同用于登录密码校验。管理员账号固定在服务端配置中，不写入该表。users 表结构见表 2.2。

表 2.2 users 表结构			
字段名	数据类型	约束	说明
id	INTEGER	主键，自增	普通用户编号
username	TEXT	非空、唯一	用户名
password_hash	TEXT	非空	密码摘要
salt	TEXT	非空	随机盐

（2）places 表。places 表用于保存红色文化景点资料。id 为自增主键，name、location 和 history 均为必填字段；景点打卡次数由 records 表实时统计，不在本表保存重复字段。places 表结构见表 2.3。

表 2.3 places 表结构

字段名	数据类型	约束	说明
id	INTEGER	主键, 自增	景点编号
name	TEXT	非空	景点名称
location	TEXT	非空	景点位置
history	TEXT	非空	历史介绍

(3) records 表。records 表用于保存普通用户的学习打卡记录。id 为自增主键, user_id 和 place_id 分别引用 users 表和 places 表, created_at 记录打卡时间, reflection 保存学习心得。用户积分和景点打卡次数均由该表记录数量计算。records 表结构见表 2.4。

表 2.4 records 表结构

字段名	数据类型	约束	说明
id	INTEGER	主键, 自增	记录编号
user_id	INTEGER	外键、非空	普通用户编号
place_id	INTEGER	外键、非空	景点编号
created_at	TEXT	非空	打卡时间
reflection	TEXT	非空	学习心得

(4) notices 表。notices 表用于保存系统公告。id 为自增主键, title、content 和 published_at 不能为空, remark 用于记录补充说明, 未填写时保存为空字符串。notices 表结构见表 2.5。

表 2.5 notices 表结构

字段名	数据类型	约束	说明
id	INTEGER	主键, 自增	公告编号
title	TEXT	非空	公告标题
content	TEXT	非空	公告内容
published_at	TEXT	非空	发布时间
remark	TEXT	非空, 默认''	备注

第三章 系统实现

3.1 功能实现

3.1.1 注册与登录功能

系统启动后首先显示登录页，用户需要确认服务端地址，再填写用户名和密码。没有账号时可以点击“注册新用户”打开图 3.1 所示窗口，依次输入用户名、密码和确认密码。这个窗口只负责收集数据，用户名是否重复、是否占用了管理员保留名称以及两次密码是否一致，都由服务端再次检查。注册成功后用户返回登录页继续操作。把校验放在服务端能够保证本机和局域网客户端使用同一套规则，下面的 service.py 代码与该页面的提交操作相对应。

●●● 注册用户

用户名

密码

确认密码

提交

图 3.1 用户注册界面

对应代码：service.py

```
if action == 'register':
    username = text_field(data, 'username', 30)
    if username == ADMIN_USERNAME:
        raise ValueError('该用户名为管理员专用账号')
    password = read_password(data)
    if password != data.get('confirm_password'):
        raise ValueError('两次密码不一致')
    salt = secrets.token_hex(16)
    db.execute(
        'INSERT INTO users(username,password_hash,salt) VALUES(?,?,?)',
        (username, password_hash(password, salt), salt)
    )
    return {'message': '注册成功. 请登录'}
```

注册请求由业务层统一处理。系统先检查管理员保留用户名和确认密码，再生成随机盐并保存密码摘要；如果用户名已经存在，数据库唯一约束会阻止重复写入。该处理过程与图 3.1 中的“提交”按钮相对应。

3.1.2 普通用户功能

普通用户登录后默认进入景点与打卡页面。图 3.2 中的上方区域提供编号和景点名称查询，中央表格显示景点名称、位置、打卡次数和热门状态，选中某一行后，下方区域会显示更完整的红色历史介绍。用户确认学习内容后，可以直接进入打卡窗口。这里展示的打卡次数并不是固定写在景点表中的数值，而是服务端查询 places 表时同时统计 records 表得到的，这与第二章的数据关系设计保持一致。对应代码完成景点筛选、次数统计和热门判断。



图 3.2 普通用户景点列表界面

对应代码：service.py

```
if action in ('list_places', 'hot_places'):
    number = read_id(data, optional=True)
    keyword = text_field(data, 'keyword', required=False)
    rows = db.execute('''SELECT p.*, COUNT(r.id) AS checkins
        FROM places p LEFT JOIN records r ON r.place_id=p.id
        WHERE (? IS NULL OR p.id=?) AND instr(p.name,?)>0
        GROUP BY p.id ORDER BY checkins DESC, p.id ASC''',
        (number, number, keyword))
    places = [dict(row) for row in rows]
    for place in places:
        place['is_hot'] = place['checkins'] >= 10
```

查询语句将景点表与学习记录表连接，按记录数量得到每个景点的打卡次数，并按照“打卡次数不少于 10 次”的规则判断热门景点。查询结果返回客户端后，直接填入图 3.2 中的列表。

用户在景点列表中选中目标景点并点击打卡按钮后，系统弹出心得填写窗口，如图 3.3 所示。窗口标题会带出当前景点名称，用户填写学习内容后再提交，能够减少景点编号输入错误。请求到达服务端后，系统会再次确认当前会话属于普通用户、景点编号真实存在，并限制心得长度。写入成功后，页面重新读取个人积分、景点打卡次数和学习记录，使三个位置显示的数据保持一致。图下代码给出了 records 表新增记录的主要处理过程。

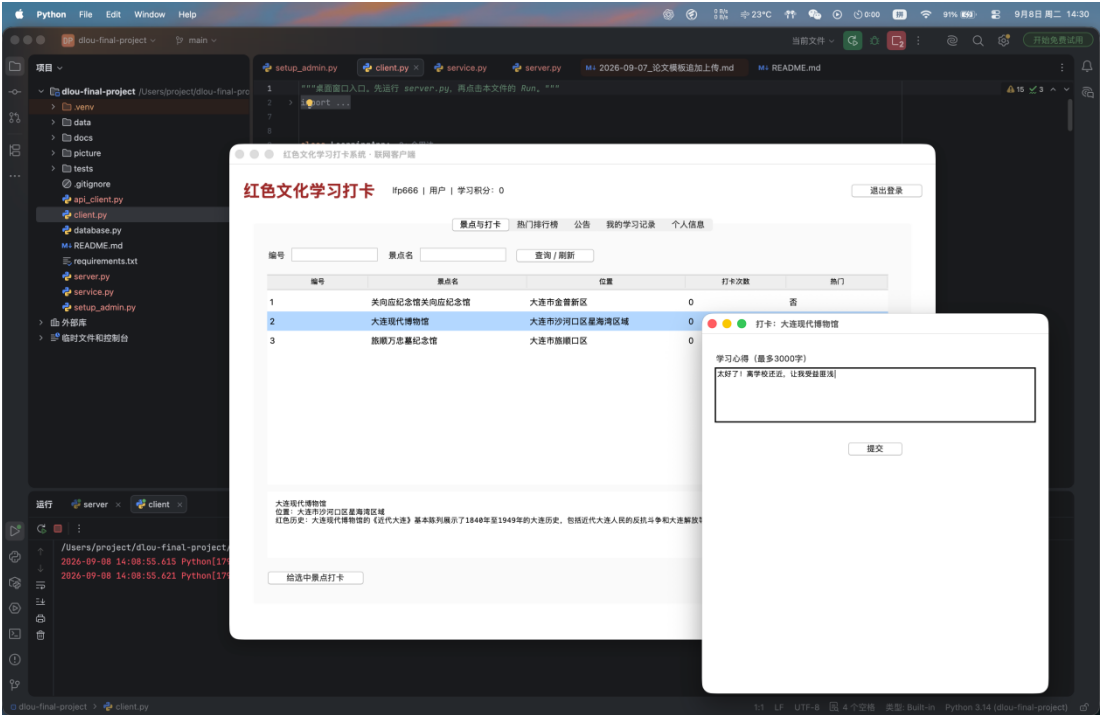


图 3.3 普通用户学习打卡界面

对应代码：service.py

```
if action == 'checkin':
    if is_admin:
        raise PermissionError('管理员不参与学习打卡')
    place_id = read_id(data, 'place_id')
    reflection = text_field(data, 'reflection', 3000)
    if not db.execute(
        'SELECT id FROM places WHERE id=?', (place_id,)).fetchone():
        raise ValueError('景点不存在')
    db.execute('INSERT INTO records
        (user_id,place_id,created_at,reflection) VALUES(?,?,?,?)',
        (user_id, place_id, now(), reflection))
    return {'message': '打卡成功, 学习积分 +1'}
```

打卡功能只允许普通用户执行，写入前还会确认景点存在并检查心得内容。记录成功写入 records 表后，个人积分和景点打卡次数会在页面刷新时按记录数量重新计算。

3.1.3 管理员功能

管理员和普通用户共用登录入口，但登录成功后显示的功能范围不同。图 3.4 保留景点、热门排行和公告等公共页面，同时增加用户列表以及景点、公告的维护按钮；管理员可以查看普通用户当前的学习积分，但不显示个人打卡记录，也不能参与学习打卡。客户端根据服务端返回的 role 调整页面，只是为了让操作更清楚，管理请求到达服务端后仍然会检查 is_admin。对应代码展示了管理员用户列表选项卡的创建过程。



图 3.4 管理员主界面

对应代码：client.py

```
if self.user['role'] == 'admin':
    frame = ttk.Frame(tabs, padding=12)
    tabs.add(frame, text='用户列表')
    self.tab_kinds.append('users')
    self.build_list(frame, 'users', [
        ('id', '编号'), ('username', '用户名'),
        ('role', '身份'), ('points', '积分')
    ])
})
```

客户端在管理员登录后增加用户列表选项卡，用于展示普通用户信息和实时积分。这里的页面差异便于操作，真正的管理员权限仍由服务端校验，不能只依靠隐藏按钮。

管理员进入公告选项卡后，可以按公告编号或标题进行查询，并通过下方按钮新增、修改或删除公告。图 3.5 展示了公告列表和管理按钮的位置，选中公告后还可以在详情区域查看完整内容和备注。新增时系统自动记录发布时间，修改时保留原编号并更新标题、内容和备注；删除前客户端会再次询问，避免误操作。图下的 service.py 代码根据是否传入公告编号区分新增和修改，并与第二章 notices 表中的字段一一对应。



图 3.5 管理员公告管理界面

对应代码：service.py

```
if action == 'save_notice':
    fields = (
        text_field(data, 'title', 100),
        text_field(data, 'content', 5000),
        text_field(data, 'remark', 500, False)
    )
    number = read_id(data, optional=True)
    if number:
        db.execute('UPDATE notices
            SET title=?,content=?,remark=? WHERE id=?', fields + (number,))
    else:
        db.execute('INSERT INTO notices
            (title,content,remark,published_at) VALUES(?,?,?,?)',
            fields + (now(),))
    return {'message': '公告已保存'}
```

服务端根据是否传入公告编号区分新增和修改操作，并对标题、内容和备注长度进行统一检查。保存完成后，客户端重新读取公告列表，所以图 3.5 能够及时显示最新内容。

3.1.4 局域网运行功能

为了验证系统不是只能在开发电脑上运行，本项目还在同一局域网中的其他电脑上启动客户端。客户端登录页把默认的 127.0.0.1 改为服务端电脑的局域网 IP，登录成功后能够读取相同的景点、公告和学习记录。图 3.6 显示了另一台电脑连接后的景点页面，列表内容与服务端数据库保持一致。整个过程中客户端不复制数据库，只通过 ApiClient 发送请求，因此后续修改服务地址时不需要改动页面业务代码。图下代码说明了请求地址、JSON 数据和会话令牌的封装方式。

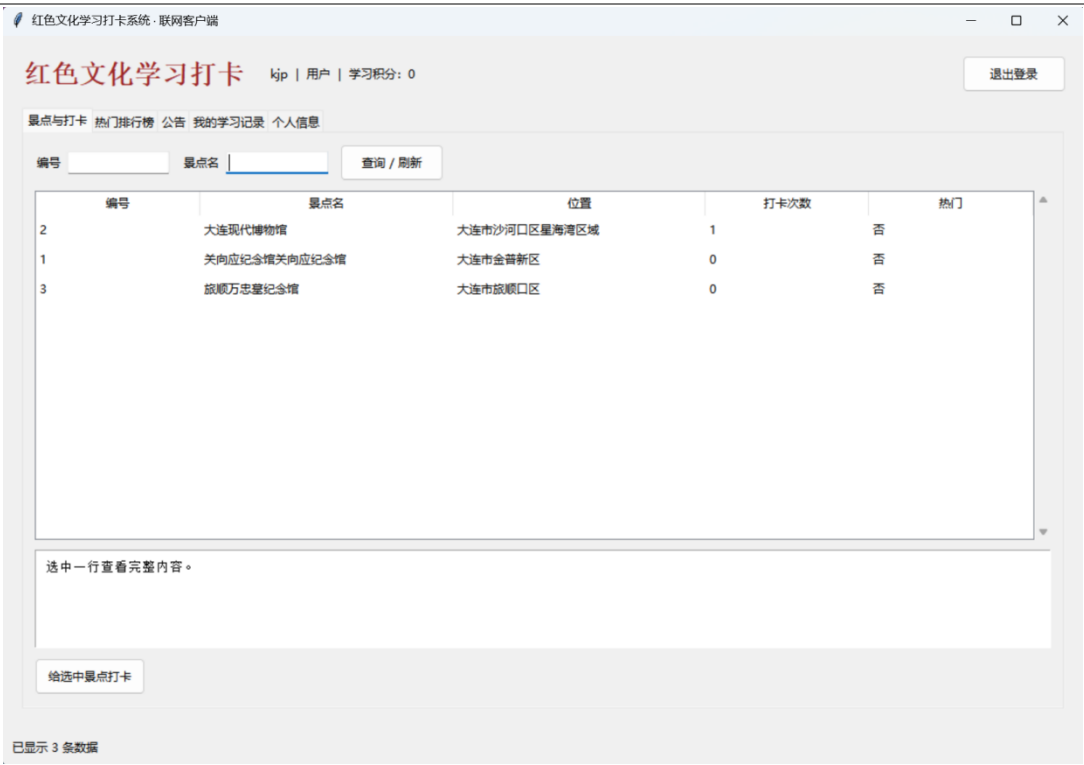


图 3.6 同一局域网客户端运行界面

对应代码: api_client.py

```
class ApiClient:
    def __init__(self, base_url='http://127.0.0.1:8765'):
        self.base_url = base_url.rstrip('/')
        self.token = ''

    def call(self, action, data=None):
        raw = json.dumps({
            'action': action, 'data': data or {}
        }).encode('utf-8')
        request = Request(self.base_url + '/api', data=raw, headers={
            'Content-Type': 'application/json',
            'Authorization': 'Bearer ' + self.token})
        return self.send(request)
```

ApiClient 把业务动作和参数编码为 JSON，并把登录令牌放入请求头。修改 base_url 后，同一套客户端既可以访问本机服务端，也可以访问图 3.6 中的局域网服务端。

图 3.7 记录了局域网客户端填写服务端地址并完成连接检查的结果。服务端默认监听 0.0.0.0:8765，本机仍可以通过 127.0.0.1 访问，其他电脑则使用服务端实际局域网 IP。连接成功只能说明两台电脑之间的地址和端口可用，登录、查询和打卡仍要继续经过正常的身份验证和业务处理。下面的 server.py 代码给出了监听地址、端口和持续提供服务的实现，它与图 2.2 中的物理部署方式相对应。

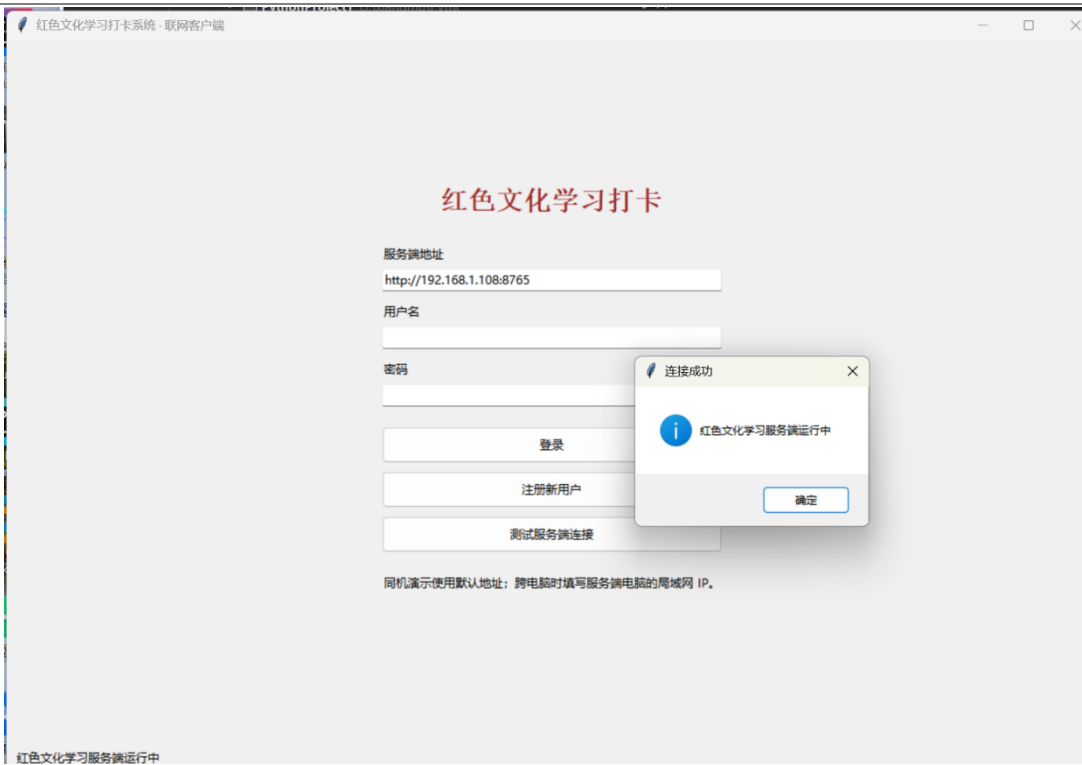


图 3.7 跨电脑服务连接测试

对应代码：server.py

```
def main():
    service = LearningService()
    host = os.environ.get('LEARNING_HOST', '0.0.0.0')
    port = int(os.environ.get('LEARNING_PORT', '8765'))
    server = make_server(service, host, port)
    print(f'服务端已启动，监听地址：{host}:{port}')
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print('服务端已停止。')
    finally:
        server.server_close()
```

服务端监听地址和端口可以通过环境变量调整。默认监听所有网卡，能够同时满足本机演示和同一局域网跨电脑测试；程序退出时会关闭服务端连接。

第四章 系统测试

4.1 Postman 接口与 API 测试

4.1.1 测试过程与结果

Postman 集合覆盖健康检查、普通用户注册与登录、公告和景点查询、学习打卡、记录管理、资料修改、管理员登录及内容维护。集合运行共执行 23 个请求和 48 项断言，48 项通过，失败 0 项，错误 0 项，总耗时 1.373 秒，平均响应时间 5 毫秒。测试结果见表 4.1。

表 4.1 Postman 接口测试结果				
测试内容	请求或断言	预期结果	实际结果	结论
接口集合	23 个请求，48 项断言	所有断言通过	48 通过，0 失败，0 错误；总耗时 1.373 s，平均 5 ms	通过
错误密码登录	POST /api, action=login	返回 401 和错误提示	401 Unauthorized; 用户名或密码错误	通过

4.1.2 测试截图

接口测试先使用 Postman 集合按业务顺序执行健康检查、注册登录、公告与景点查询、学习打卡、记录维护以及管理员操作。图 4.1 左侧列出了本次运行的请求，右侧显示各请求和断言的执行状态，底部汇总给出请求数量、断言数量、耗时和平均响应时间。23 个请求共包含 48 项断言，本次运行中没有出现失败或错误。该结果主要用于确认正常业务流程中的请求参数、状态码和返回数据能够前后衔接。

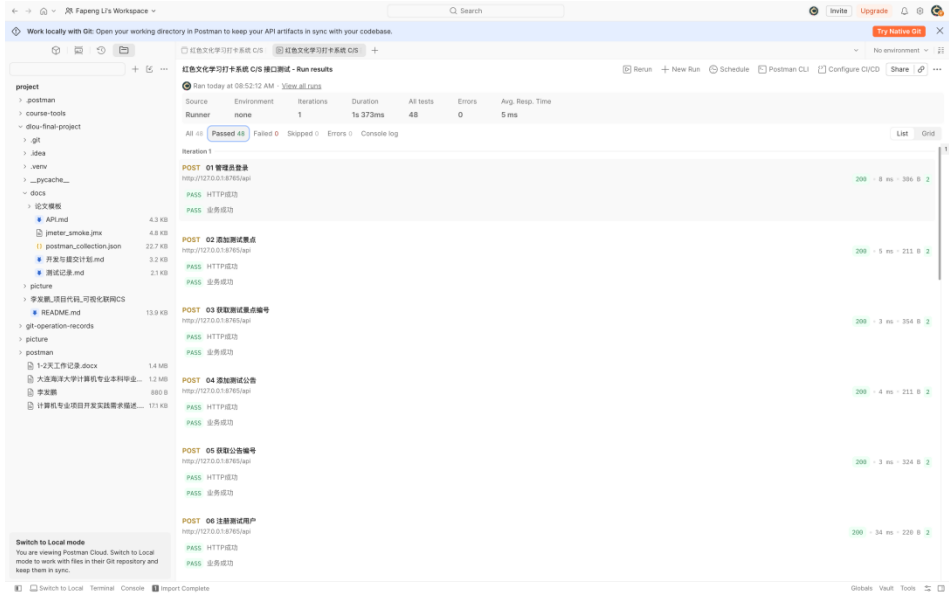


图 4.1 Postman 集合运行结果

正常流程通过后，又单独使用错误密码提交登录请求，用来检查系统是否会把异常登录误判

为成功。图 4.2 中服务端返回 401 Unauthorized，响应内容为“用户名或密码错误”，没有返回用户信息和会话令牌。这个结果与登录接口的预期处理一致，也说明客户端收到同类响应时能够显示明确提示。将该用例与图 4.1 的正常登录结果对照，可以同时验证登录接口的成功分支和失败分支。

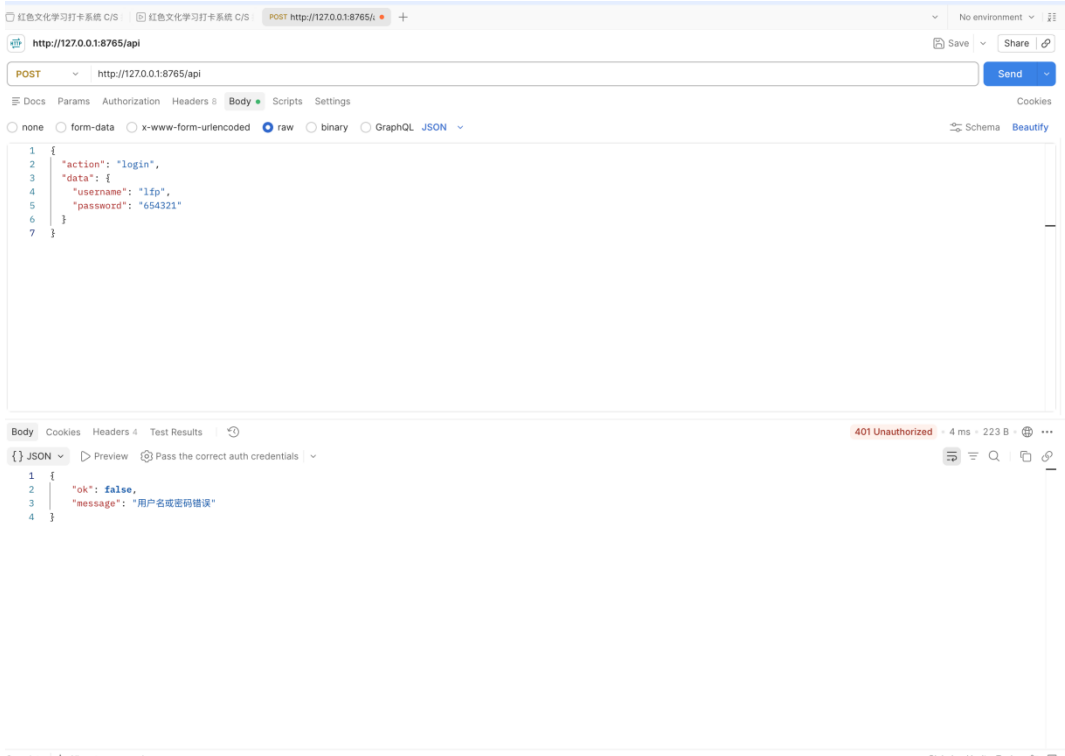


图 4.2 Postman 错误密码登录测试

4.2 JMeter 性能与压力测试

4.2.1 测试方案

接口功能确认后，使用 JMeter 对本机服务端进行轻量并发测试。图 4.3 中的线程数设置为 5，启动时间设置为 2 秒，每个线程循环 10 次，因此总共发送 50 个请求。选择较小的并发量，是因为本次测试主要观察课程项目在短时间连续访问时是否出现报错，而不是模拟互联网中的大规模用户。测试请求使用已经准备好的只读接口，能够减少写入数据变化对响应时间的影响。

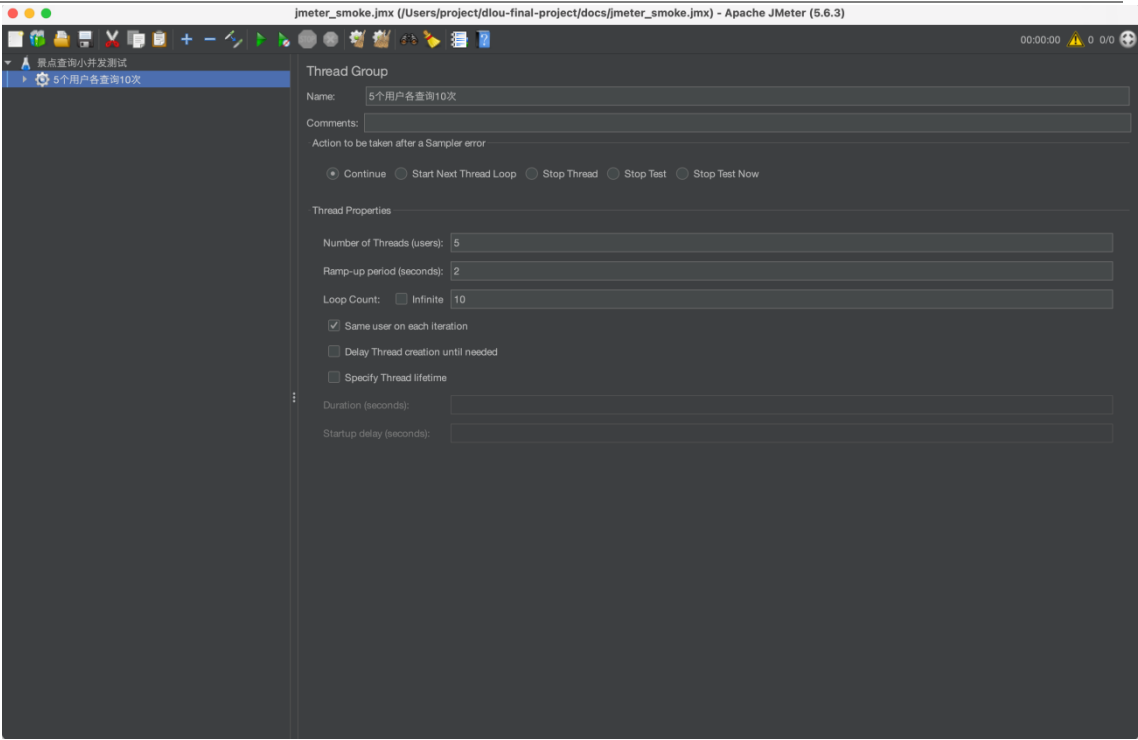


图 4.3 JMeter 线程组参数

4.2.2 测试结果与分析

汇总报告显示 50 个样本全部完成，平均响应时间 3 毫秒，最小 1 毫秒，最大 31 毫秒，标准差 4.13 毫秒，错误率 0.00%，吞吐量 31.1 次/秒。具体结果见表 4.2。

表 4.2 JMeter 性能测试结果

样本数	平均响应时间	最小/最大	错误率	吞吐量
50	3 ms	1 ms / 31 ms	0.00%	31.1 次/秒

JMeter 执行完成后，汇总报告从样本数、平均响应时间、最小值、最大值、标准差、错误率和吞吐量几个方面显示结果。图 4.4 中 50 个样本全部完成，平均响应时间为 3 毫秒，最小 1 毫秒，最大 31 毫秒，错误率为 0.00%，吞吐量为 31.1 次/秒。结合线程组参数可以看出，本机轻量负载下没有出现请求失败。这里的数据只反映本次设备、接口和网络条件，不能直接代表系统在公网或高并发环境中的性能。

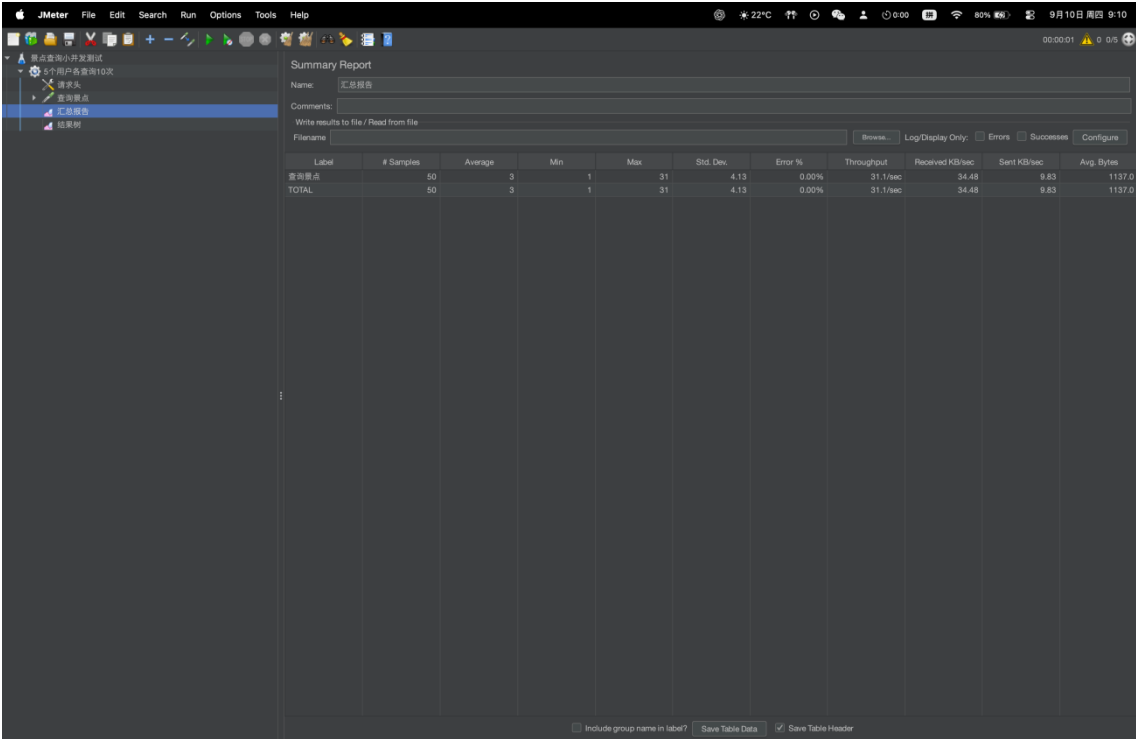


图 4.4 JMeter 汇总报告

汇总数据只能说明整体情况，还需要通过结果树检查单个请求。图 4.5 左侧的样本均显示绿色成功状态，选中某个样本后，可以在右侧查看请求地址、请求方法、响应状态和耗时。逐条检查能够发现汇总报告中不容易看出的参数错误或异常响应。本次查看的样本状态与汇总报告一致，没有出现连接失败和服务端错误。

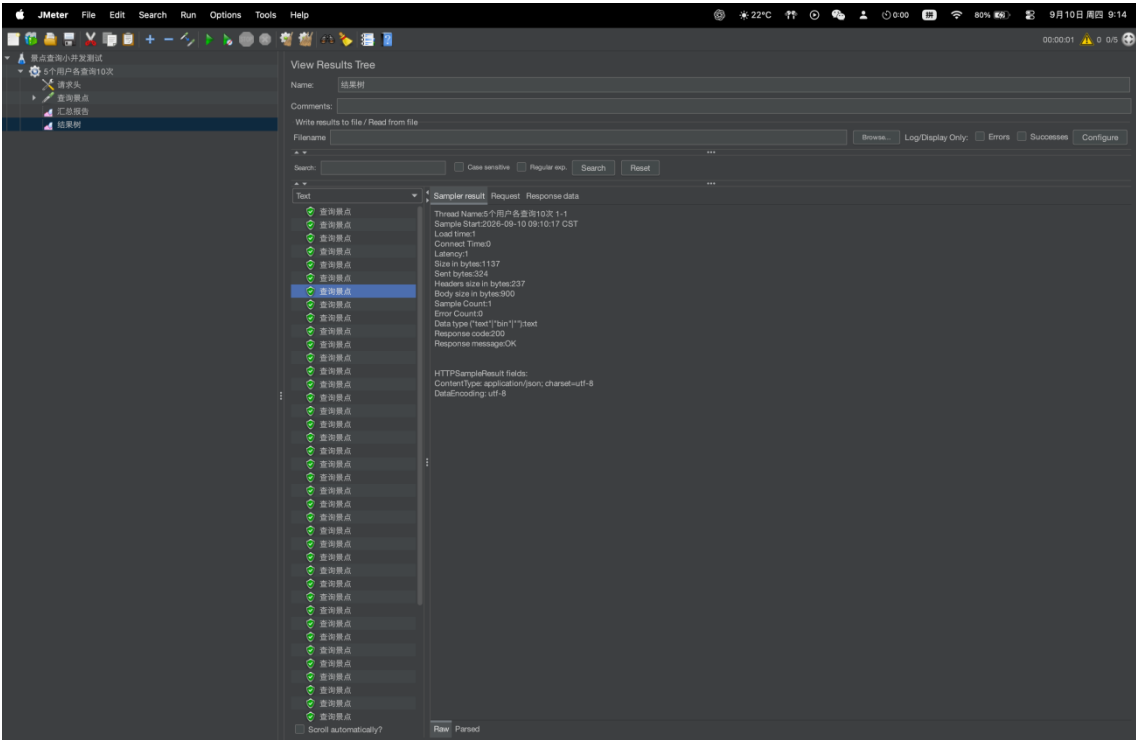


图 4.5 JMeter 查看结果树

在结果树中切换到响应数据区域，可以进一步查看服务端返回的 JSON 内容。图 4.6 展示的样本不仅返回了成功状态，还包含接口对应的数据字段，说明请求被服务端正确解析并完成了业务处理。把响应内容与 Postman 中的接口结果进行对照，可以确认两种测试工具访问的是同一套接口。本图作为 JMeter 测试的最后一步，补充说明了“请求成功”不仅是状态码正常，返回的数据内容也符合预期。

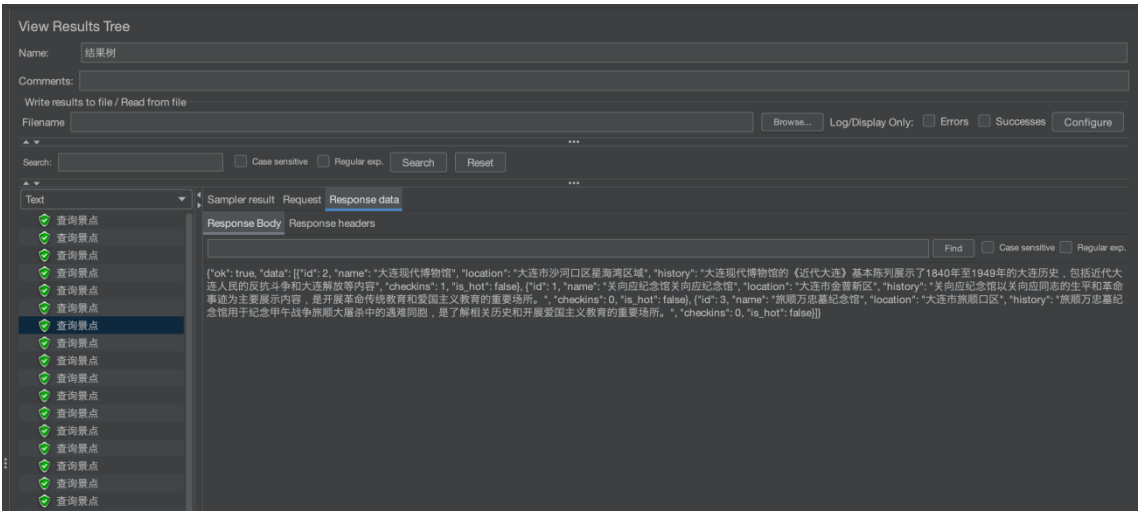


图 4. 6 JMeter 响应数据